

Assignment 1 Theory Problem Set

DO NOT TAG

Name: Edward Capone
GT Email: ecapone3@gatech.edu

Theory PS Q1

Given the softmax function:

$$s_i = \frac{e^{z_i}}{\sum_k e^{z_k}}$$

The input vector z is called logits. Our Goal is to derive the gradient of s with respect to z , i.e.:

$$\frac{\partial s_i}{\partial z_j}$$

To differentiate the softmax function, consider the quotient rule:

$$\frac{\partial s_i}{\partial z_j} = \frac{\partial}{\partial z_j} \left(\frac{e^{z_i}}{\sum_k e^{z_k}} \right)$$

When we apply the derivative, we get:

$$\begin{aligned} \frac{\partial s_i}{\partial z_j} &= \frac{e^{z_i} \delta_{ij} \sum_k e^{z_k} - e^{z_i} e^{z_j}}{(\sum_k e^{z_k})^2} \\ &= s_i (\delta_{ij} - s_j) \end{aligned}$$

Hence, the gradient of the softmax function is:

$$\frac{\partial s_i}{\partial z_j} = \begin{cases} s_i(1 - s_i) & \text{if } i = j \\ -s_i s_j & \text{if } i \neq j \end{cases}$$

The result forms a Jacobian matrix of the softmax function:

$$\frac{\partial \mathbf{s}}{\partial \mathbf{z}} = \begin{bmatrix} s_1(1-s_1) & -s_1s_2 & \cdots & -s_1s_n \\ -s_2s_1 & s_2(1-s_2) & \cdots & -s_2s_n \\ \vdots & \vdots & \ddots & \vdots \\ -s_ns_1 & -s_ns_2 & \cdots & s_n(1-s_n) \end{bmatrix}$$

This Jacobian matrix captures both the diagonal and off-diagonal elements, which are critical for understanding how the softmax function changes with respect to logits.

Theory PS Q2

We are given a linear threshold neuron with the function:

$$f(\mathbf{x}) = \begin{cases} 1 & \text{if } \mathbf{w}^T \mathbf{x} + b \geq 0 \\ 0 & \text{if } \mathbf{w}^T \mathbf{x} + b < 0 \end{cases}$$

We need to find weight vectors $\mathbf{w}$ and b for the AND and OR function.

1. AND Function

The truth table for the AND function is:

x_1	x_2	$f_{\text{AND}}(x)$
0	0	0
0	1	0
1	0	0
1	1	1

The decision boundary equation is:

$$w_1x_1 + w_2x_2 + b \geq 0$$

To satisfy the AND function, a possible solution is:

$$w_{\text{AND}} = [1, 1], \quad b_{\text{AND}} = -1.5$$

- For $(x_1, x_2) = (1, 1)$: $1(1) + 1(1) - 1.5 = 0.5 \geq 0$ (Output 1)
- For all other input, the result is < 0 , leading to output 0

2. OR Function

The truth table for the AND function is:

x_1	x_2	$f_{\text{OR}}(x)$
0	0	0
0	1	1
1	0	1
1	1	1

The decision boundary equation is:

$$w_1x_1 + w_2x_2 + b \geq 0$$

To satisfy the OR function, a possible solution is:

$$w_{\text{OR}} = [1, 1], \quad b_{\text{OR}} = -0.5$$

- For $(x_1, x_2) = (0,1)$ or $(1,0)$ or $(1,1)$ the result is ≥ 0 (Output 1)
- For $(x_1, x_2) = (0,0)$: the result < 0 , which leads to an output of 0

Theory PS Q3

The XOR function is defined as follows:

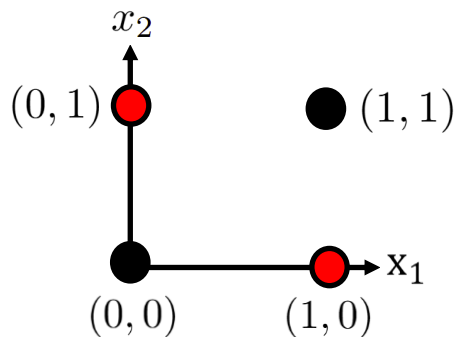
x_1	x_2	$f_{\text{XOR}}(x)$
0	0	0
0	1	1
1	0	1
1	1	0

We need to prove that XOR cannot be represented using a linear model of the form:

$$f(\mathbf{x}) = \begin{cases} 1 & \text{if } \mathbf{w}^T \mathbf{x} + b \geq 0 \\ 0 & \text{if } \mathbf{w}^T \mathbf{x} + b < 0 \end{cases}$$

To visualize the XOR function, we plot the points on a 2D plane:

$$(0, 0) \rightarrow 0, \quad (0, 1) \rightarrow 1, \quad (1, 0) \rightarrow 1, \quad (1, 1) \rightarrow 0$$



The points belonging to the same class (output = 1) are diagonally opposite and cannot be separated by a single linear boundary.

Assume that there exists a linear model:

$$w_1x_1 + w_2x_2 + b \geq 0$$

When we substitute the XOR truth values, we get:

$$w_1(0) + w_2(0) + b < 0 \Rightarrow b < 0$$

$$w_1(1) + w_2(0) + b \geq 0 \Rightarrow w_1 + b \geq 0$$

$$w_1(0) + w_2(1) + b \geq 0 \Rightarrow w_2 + b \geq 0$$

$$w_1(1) + w_2(1) + b < 0 \Rightarrow w_1 + w_2 + b < 0$$

From the inequalities in the previous slide, if we add the first and last equations, we come up with:

$$b + (w_1 + w_2 + b) < 0 \quad \Rightarrow \quad 2b + w_1 + w_2 < 0$$

This contradicts the previous conditions ($w_2 + b \geq 0$) and ($w_1 + b \geq 0$), proving that no such linear boundary can satisfy all conditions simultaneously.

Therefore, the XOR function is **not linearly separable**. To correctly model XOR, we need a non-linear transformation or an additional hidden layer in a neural network.

Assignment 1 Paper Review

DO NOT TAG

Name: Edward Capone

GT Email: ecapone3@gatech.edu

Paper 1 Review

Main Contributions:

The paper "Weight Agnostic Neural Networks (WANNs)" introduces a novel approach to neural network design that challenges traditional weight-optimized models by demonstrating that certain architectures can achieve meaningful performance even with fixed, shared weights. The key contribution of this work is the idea that neural architectures themselves possess significant representational power, independent of the specific weight values. The authors employ an evolutionary search algorithm to identify network architectures capable of performing well across multiple tasks without extensive weight tuning. This approach provides insights into the balance between innate structure versus learned parameters, drawing parallels to biological learning systems. One of the major strengths of the paper is its innovative perspective, which challenges conventional deep learning paradigms by shifting focus from weight optimization to architectural discovery. The experiments demonstrate the versatility of WANNs across reinforcement learning and supervised learning tasks, suggesting broader applicability. Additionally, the paper offers compelling visualizations and intuitive explanations, making complex concepts accessible to a wider audience. However, a notable weakness is that WANNs currently underperform compared to traditional architectures, such as convolutional neural networks (CNNs), which are finely tuned through gradient-based optimization. Furthermore, the evolutionary search process used to identify effective architectures can be computationally expensive, posing practical challenges in large-scale applications.

Personal Takeaway:

The paper presents a fresh perspective on neural network design by emphasizing the role of architecture over weight optimization. This shift opens several exciting avenues for future research, such as developing novel neural architectures optimized for weight-agnostic conditions and exploring how adaptive mechanisms could further enhance performance. Additionally, the cross-disciplinary approach that integrates neuroscience insights into AI research presents opportunities to develop models that better mimic biological learning systems. A key takeaway is that while the current performance of WANNs may be limited, the potential to develop networks that require minimal tuning could lead to significant advancements in resource-efficient AI systems, particularly in scenarios where traditional training is infeasible. Overall, the paper encourages a rethink of how we approach neural network design, emphasizing structure as a key driver of intelligence.

Paper 1: Question 1

The traditional view of optimization in deep learning focuses on searching for the best weights to minimize loss. However, the paper introduces a paradigm shift by emphasizing the search for network architectures over weight optimization, redefining the search problem in neural networks. Instead of optimizing weights, the paper proposes exploring neural architectures that perform well across tasks even with fixed or shared weights. This weight-agnostic approach broadens the search space by evaluating architectures based on their inherent representational power, enabling a focus on innate design rather than parameter tuning.

The paper employs a population-based evolutionary search, where minimal neural topologies are evolved and evaluated using a shared weight. The tournament selection process ensures that high-performing and robust architectures survive for further exploration, mimicking natural selection. This evolutionary strategy avoids the computational cost of traditional weight optimization by bypassing gradient descent entirely. By using fixed weights, the paper simplifies the evaluation process, reducing the computational burden typically associated with training. This efficiency allows for rapid exploration of architecture space and enables the discovery of structures that generalize well across diverse tasks. The emphasis on architecture inherently ties to the biological parallel of innate versus learned behaviors, where neural architectures act as innate structures that guide performance without extensive weight training.

This perspective challenges the conventional approach to optimization, encouraging the discovery of architectures with intrinsic capabilities. It also opens opportunities for developing novel network designs, shifting the focus from refining weights to innovating in architecture. As a result, this approach contributes to more efficient, adaptable, and robust AI systems. By shifting the focus from weight optimization to architecture search, the paper presents an alternative approach that could lead to advancements in AI by reducing computational costs while discovering powerful network topologies. Overall, the search process described in the paper redefines learning as a structural exploration rather than a fine-tuning problem, offering valuable insights into the role of architecture in achieving generalizable and effective performance across multiple tasks.

Paper 1: Question 2

The paper on Weight Agnostic Neural Networks (WANNs) explores the representational power of neural network architectures independent of weight training, positing that architecture alone can encode solutions for specific tasks even with randomly initialized weights. This challenges the traditional belief that optimizing weights is crucial for effective performance. The study demonstrates that certain architectures can perform tasks with minimal weight training, such as achieving significant accuracy on the MNIST dataset using a single shared weight sampled from a uniform distribution. This suggests that architecture possesses inherent representational power, offering strong inductive biases that guide task performance without extensive weight optimization. The method of weight determination does play a role, as different approaches yield varying levels of performance. While the paper highlights that random weights can lead to reasonable outcomes, fine-tuning weights further enhances performance, showing that both elements contribute to a network's success. The paper's approach of evaluating architectures with shared weights simplifies the search process and avoids the computational costs associated with traditional training methods, making it particularly useful for resource-constrained environments. Despite the strength of architectures, the study illustrates that architecture, and weights do not have equal representational power. Architecture defines the structure and potential of the network, acting as a foundation for task performance, whereas weights are crucial for refining that potential to achieve optimal results. A well-designed architecture can perform adequately with random weights, but reaching state-of-the-art performance requires precise weight optimization. The authors' evolutionary search approach reveals that certain architectures are inherently suited to specific tasks, supporting the argument that architecture can embed useful inductive biases. However, weight optimization remains essential for fine-tuning performance and improving generalization across tasks. In conclusion, the paper emphasizes that while architecture is fundamental to representational power, the interplay between architecture and weight optimization is necessary for achieving robust and adaptable AI systems. The findings encourage further exploration into architecture-driven learning, highlighting the potential of discovering minimal yet powerful network topologies without relying heavily on weight training.

Assignment 1 Writeup

DO NOT TAG

Name: Edward Capone

GT Email: ecapone3@gatech.edu

Two-Layer Neural Network

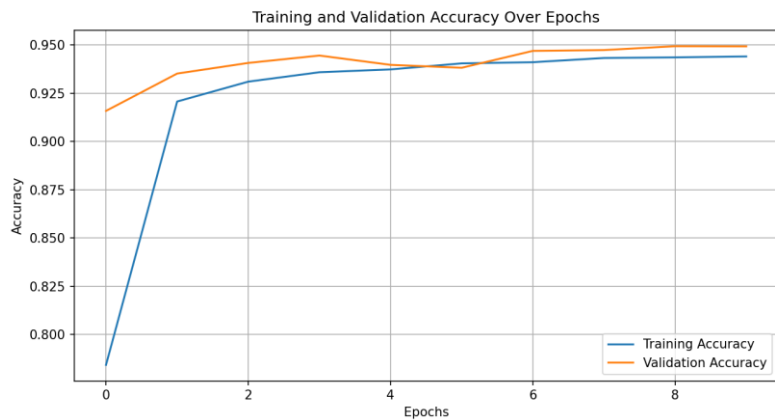
DO NOT TAG

1. Learning Rates

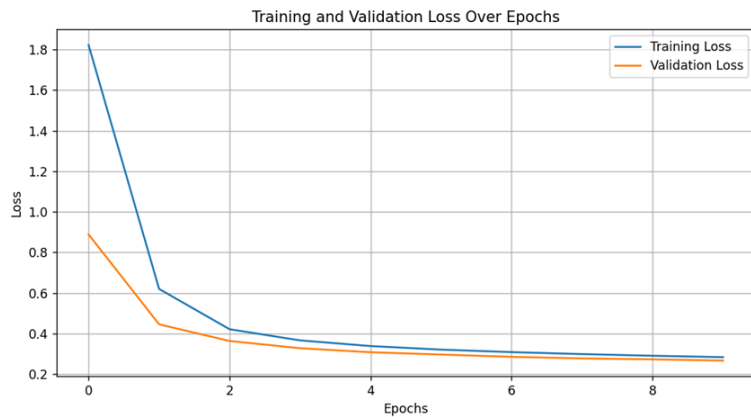
Tune the learning rate of the model with all other default hyper-parameters fixed. Fill in the table below:

	lr=1	lr=1e-1	lr=5e-2	lr=1e-2
Training Accuracy	94.41%	92.82%	91.66%	76.75%
Test Accuracy	94.78%	92.61%	91.12%	75.48%

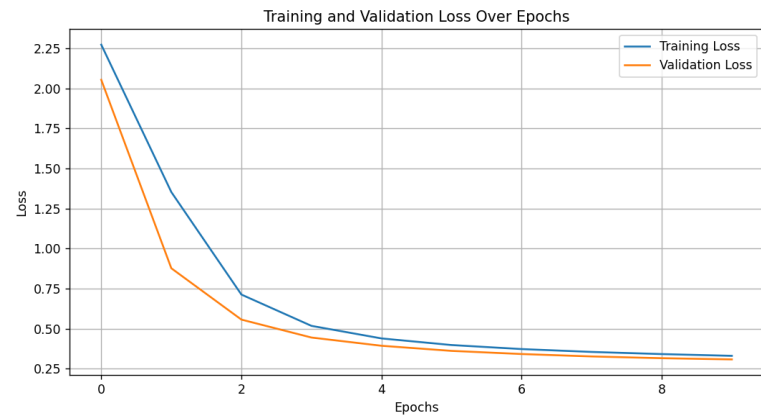
1. Learning Curve (lr = 1)



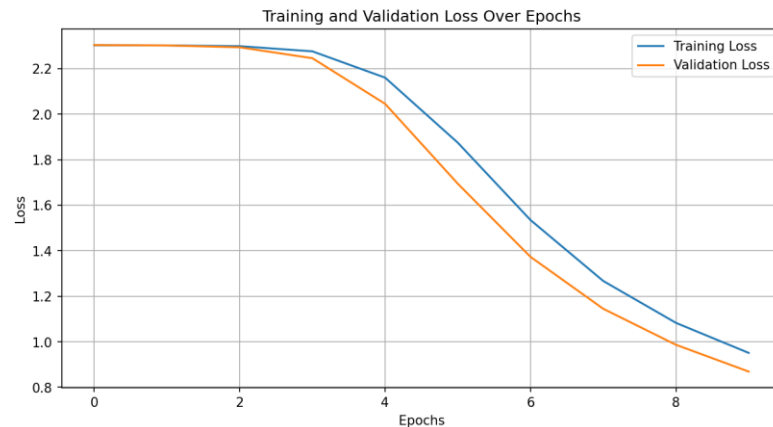
1. Learning Curve (lr = 1e-1)



1. Learning Curve (lr = 5e-2)



1. Learning Curve (lr = 1e-2)



1. Learning Rates

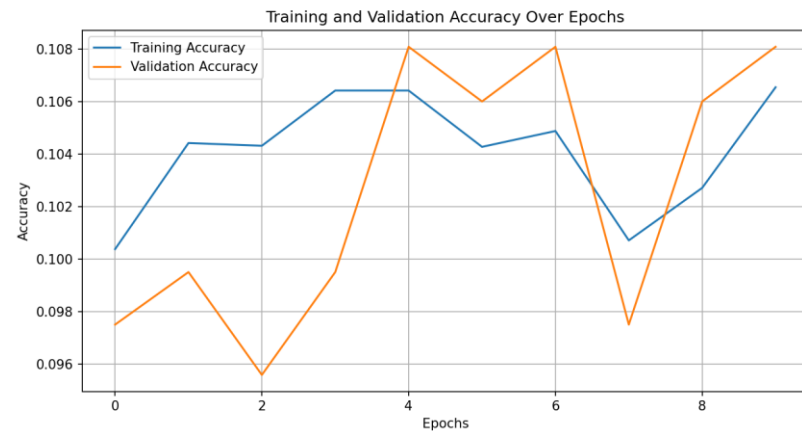
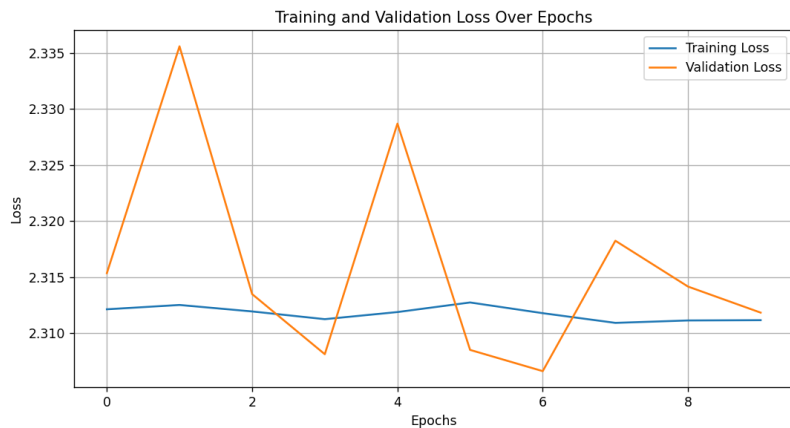
In this experiment, we tested different learning rates (1, $1e-1$, $5e-2$, $1e-2$) while keeping other hyperparameters constant to analyze their impact on training and test accuracy. A high learning rate of 1 resulted in the highest training (94.41%) and test accuracy (94.78%), indicating rapid convergence without significant overshooting. However, excessively high learning rates can risk instability and divergence. Moderate learning rates of $1e-1$ and $5e-2$ achieved stable convergence with slightly lower accuracy (92.82%, 92.61% and 91.66%, 91.12% respectively), highlighting a trade-off between speed and precision. A low learning rate of $1e-2$ resulted in the poorest performance (76.75%, 75.48%), suggesting slow convergence and potential underfitting within the given number of epochs. These results demonstrate that learning rates significantly impact model performance, requiring a balance to avoid overshooting or slow training. Future improvements could include using learning rate schedules and testing more values to achieve optimal performance.

2. Regularization

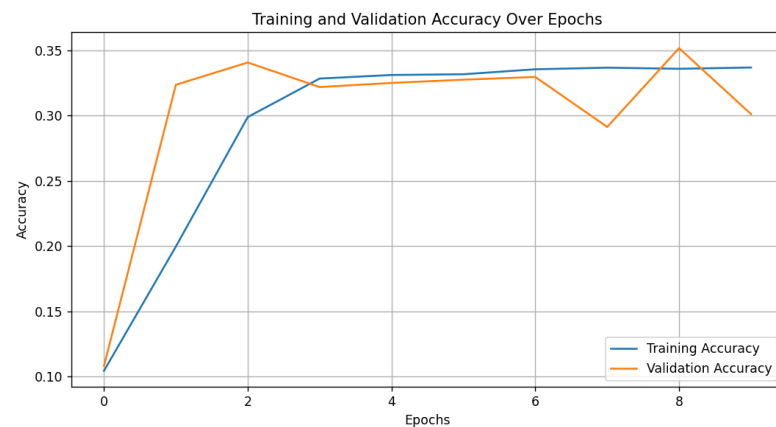
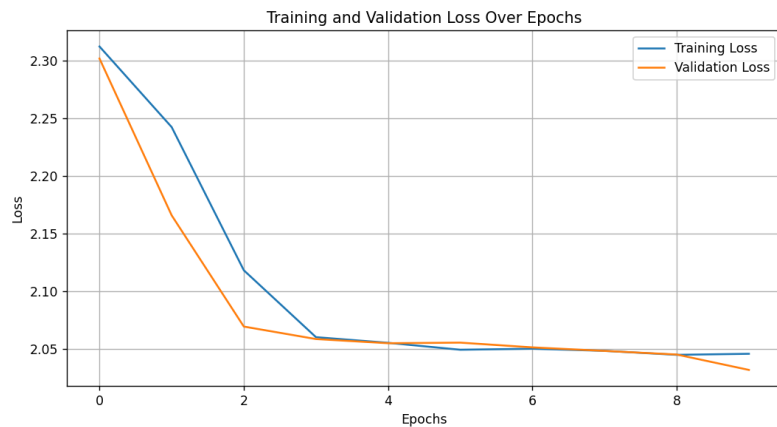
Tune the regularization coefficient of the model with all other default hyperparameters fixed. Fill in the table below:

	alpha=1	alpha=1e-1	alpha=1e-2	alpha=1e-3	alpha=1e-4
Training Accuracy	10.44%	33.69%	88.48%	92.09%	92.28%
Validation Accuracy	10.60%	30.12%	89.50%	92.67%	93.33%
Test Accuracy	10.28%	35.27%	89.25%	92.50%	93.33%

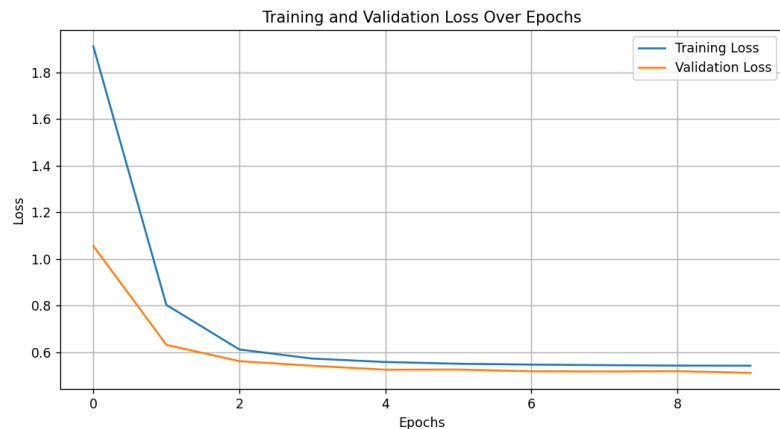
2. Regularization ($\alpha = 1$)



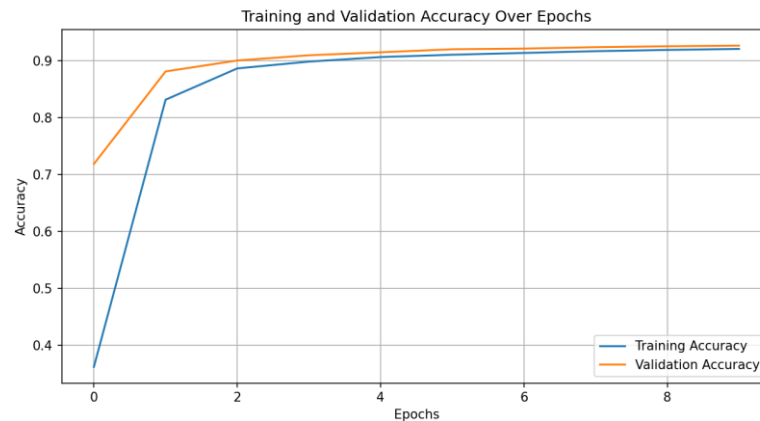
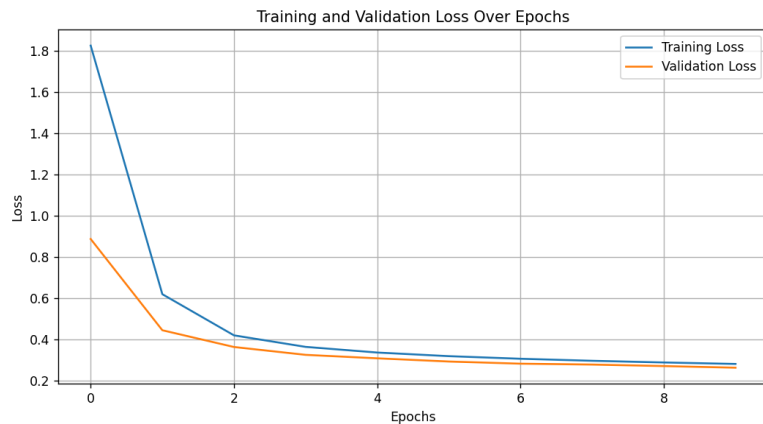
2. Regularization ($\alpha = 1e-1$)



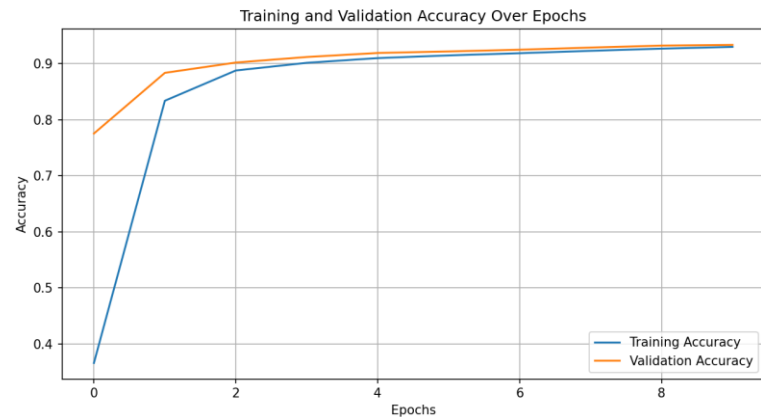
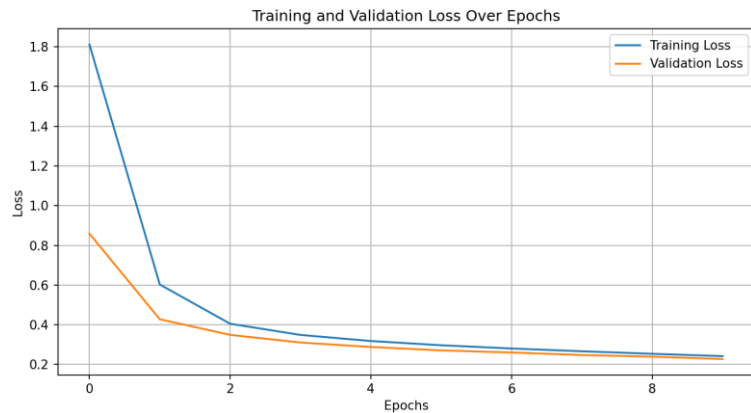
2. Regularization ($\alpha = 1e-2$)



2. Regularization ($\alpha = 1e-3$)



2. Regularization ($\alpha = 1e-4$)



2. Regularization

The results from tuning the regularization coefficient (α) demonstrate the crucial role of regularization in balancing bias and variance in the model. Regularization is applied to prevent overfitting by penalizing large weights, encouraging simpler models that generalize better to unseen data. From the table, it is evident that with a high regularization value ($\alpha=1$), the model severely underfits the data, leading to low training, validation, and test accuracy. This occurs because the regularization term dominates the loss function, overly constraining the model and preventing it from learning complex patterns in the data. As the regularization value decreases ($\alpha=1e-2$ to $\alpha=1e-4$), the model achieves significantly higher accuracy, indicating better learning capacity with reduced constraints on the weights. However, when α is too small, such as $1e-4$, the training accuracy is very high, suggesting potential overfitting, as the model may fit noise in the training data, leading to slightly diminished generalization performance compared to $\alpha=1e-3$. The experimentation methodology involved systematically varying α while keeping other hyperparameters constant, ensuring a fair comparison of the effect of regularization alone. The best performance is observed at $\alpha=1e-3$, where the model achieves a balance between bias and variance, resulting in the highest validation and test accuracy. These findings underscore the importance of tuning regularization to achieve an optimal trade-off between model complexity and generalization.

3. Hyper-parameter Tuning

You are now free to tune any hyper-parameters for better accuracy. Create a table below and put the configuration of your best model and accuracy into the table:

Batch Size	Learning Rate	Regularization	Hidden Size	Momentum	Training Accuracy	Test Accuracy
64	0.8	1e-4	64	0.65	97.66%	96.62%

The chosen hyperparameters work effectively due to their ability to balance the trade-offs between learning speed, model complexity, and generalization. The learning rate of 0.8 allows the model to make significant progress during training without overshooting the optimal solution, ensuring fast convergence while maintaining stability. A lower regularization coefficient (1e-4) prevents overfitting by adding a small penalty to the model weights, allowing the network to generalize well to unseen data without sacrificing accuracy. The batch size of 64 strikes a balance between computational efficiency and the ability to capture the complexity of the data, leading to more frequent updates and improved generalization. The hidden size of 64 ensures the model has sufficient capacity to learn meaningful patterns while avoiding overfitting that could arise from an overly complex architecture. Additionally, a momentum value of 0.65 helps accelerate convergence by dampening oscillations and smoothing weight updates, which improves training efficiency and stability. These choices were made based on iterative experimentation, where various values were tested and evaluated against performance metrics. The expected behavior was that increasing the learning rate and adjusting regularization would enhance convergence speed and test accuracy while avoiding overfitting. The final results confirmed that the chosen configuration achieved an optimal balance, yielding the highest training and test accuracies.